

Q1.

```
import math
```

```
# Node of the tree
```

```
class Node:
```

```
    def __init__(self, key):
```

```
        self.key = key
```

```
        self.left = None
```

```
        self.right = None
```

```
class HybridTree:
```

```
    def __init__(self, k):
```

```
        self.root = None
```

```
        self.k = k
```

```
        self.insertions = 0
```

```
        self.rebuilds = 0
```

```
# 1. Normal BST insertion (NO rotations)
```

```
def insert(self, key):
```

```
    new_node = Node(key)
```

```
    if self.root is None:
```

```
        self.root = new_node
```

```
    else:
```

```
        current = self.root
```

```
        while True:
```

```
if key < current.key:
    if current.left is None:
        current.left = new_node
        break
    current = current.left

elif key > current.key:
    if current.right is None:
        current.right = new_node
        break
    current = current.right

else:
    print("Duplicate key:", key)
    return
```

```
self.insertions += 1
```

```
# Check after every k insertions
```

```
if self.insertions % self.k == 0:
```

```
    if not self.is_balanced():
```

```
        self.rebuild_to_avl()
```

```
# Calculate height of tree
```

```
def height(self, node):
```

```
    if node is None:
```

```
return 0
```

```
return 1 + max(  
    self.height(node.left),  
    self.height(node.right)  
)
```

Count number of nodes

```
def count_nodes(self, node):
```

```
    if node is None:
```

```
        return 0
```

```
    return (
```

```
        1
```

```
        + self.count_nodes(node.left)
```

```
        + self.count_nodes(node.right)
```

```
    )
```

2. Check height condition

```
def is_balanced(self):
```

```
    n = self.count_nodes(self.root)
```

```
    h = self.height(self.root)
```

```
    if n <= 1:
```

```
        return True
```



```
limit = 2 * math.log2(n)
```

```
print("Nodes:", n,  
      "Height:", h,  
      "Limit:", round(limit, 2))
```

```
return h <= limit
```

```
# Store all keys using inorder traversal
```

```
def inorder(self, node, keys):
```

```
    if node is not None:
```

```
        self.inorder(node.left, keys)
```

```
        keys.append(node.key)
```

```
        self.inorder(node.right, keys)
```

```
# 3. Build balanced AVL tree from sorted keys
```

```
def build_balanced(self, keys, start, end):
```

```
    if start > end:
```

```
        return None
```

```
    mid = (start + end) // 2
```

```
    node = Node(keys[mid])
```

```
    node.left = self.build_balanced(  
        keys, start, mid - 1
```

)

node.right = self.build_balanced(

keys, mid + 1, end

)

return node

Rebuild entire tree

def rebuild_to_avl(self):

keys = []

Extract all keys

self.inorder(self.root, keys)

Keys are already sorted by inorder traversal

keys.sort()

Build balanced tree

self.root = self.build_balanced(

keys, 0, len(keys) - 1

)

self.rebuilds += 1

print(">>> Tree rebuilt as balanced AVL tree!")

```
print("> > > Total rebuilds:", self.rebuilds)
```

```
# Display inorder traversal
```

```
def display(self):
```

```
    keys = []
```

```
    self.inorder(self.root, keys)
```

```
    print("Inorder:", keys)
```

```
k = int(input("Enter k (check after every k insertions):"))
```

```
tree = HybridTree(k)
```

```
n = int(input("Enter number of keys:"))
```

```
print("Enter", n, "keys:")
```

```
for i in range(n):
```

```
    key = int(input())
```

```
    tree.insert(key)
```

```
print("\nFinal Tree")
```

```
tree.display()
```

```
print("Final Height:", tree.height(tree.root))
```



```
print("Number of Rebuilds:", tree.rebuilds)
```

Example Input

Enter k (check after every k insertions): 3

Enter number of keys: 10

Enter 1

2

3

4

5

6

7

8

9

10

How it works

Insert Key



Normal BST Insertion



Increment Insertion Counter



Is insertion count a multiple of k?



YES



Calculate N and Height

↓

Height $> 2 \times \log_2 (N)$?

↓

YES

↓

Extract all keys

↓

Sort keys

↓

Build balanced tree

↓

Increment rebuild counter

Important functions

Function	Purpose
insert()	Performs ordinary BST insertion without rotations
height()	Calculates tree height
count_nodes()	Calculates N
is_balanced()	Checks $\text{height} \leq 2 \times \log_2 (N)$
inorder()	Extracts all keys
build_balanced()	Creates a balanced AVL-style tree
rebuild_to_avl()	Rebuilds the complete tree
rebuilds	Counts total number of rebuilds

Q2.

File System LRU Cache Using a Splay Tree

Problem Understanding

A **Splay Tree** is useful for a file-system cache because every accessed file is moved close to the root. Therefore, frequently accessed files can be accessed quickly.

However, during a **virus scan**, files may be accessed sequentially in alphabetical order:

a.txt → b.txt → c.txt → d.txt → e.txt → ...

Each access causes a **splay operation**, repeatedly changing the tree structure. This can destroy the structure that was optimized for frequently accessed files.

Simple Python Implementation

```
class Node:
```

```
    def __init__(self, key):
```

```
        self.key = key
```

```
        self.left = None
```

```
        self.right = None
```

```
class SplayTree:
```

```
    def __init__(self):
```

```
        self.root = None
```

```
    # Right rotation
```

```
    def right_rotate(self, x):
```

```
        y = x.left
```

```
        x.left = y.right
```

```
        y.right = x
```

```
        return y
```

Left rotation

```
def left_rotate(self, x):
```

```
    y = x.right
```

```
    x.right = y.left
```

```
    y.left = x
```

```
    return y
```

Splay operation

```
def splay(self, root, key):
```

```
    if root is None or root.key == key:
```

```
        return root
```

Key lies in left subtree

```
    if key < root.key:
```

```
        if root.left is None:
```

```
            return root
```

Zig-Zig

```
    if key < root.left.key:
```

```
        root.left.left = self.splay(
```

```
            root.left.left, key
```

```
        )
```

```
        root = self.right_rotate(root)
```

```
# Zig-Zag
```

```
elif key > root.left.key:
```

```
    root.left.right = self.splay(
```

```
        root.left.right, key
```

```
    )
```

```
if root.left.right is not None:
```

```
    root.left = self.left_rotate(root.left)
```

```
# Final rotation
```

```
if root.left is None:
```

```
    return root
```

```
else:
```

```
    return self.right_rotate(root)
```

```
# Key lies in right subtree
```

```
else:
```

```
if root.right is None:
```

```
    return root
```

```
# Zag-Zag
```

```
if key > root.right.key:
```

```
    root.right.right = self.splay(
```

```
        root.right.right, key
```



```

    )

    root = self.left_rotate(root)

# Zag-Zig
elif key < root.right.key:
    root.right.left = self.splay(
        root.right.left, key
    )

    if root.right.left is not None:
        root.right = self.right_rotate(root.right)

# Final rotation
if root.right is None:
    return root
else:
    return self.left_rotate(root)

# Insert file
def insert(self, key):

    if self.root is None:
        self.root = Node(key)
        return

    self.root = self.splay(self.root, key)

```

```
if self.root.key == key:
```

```
    return
```

```
new_node = Node(key)
```

```
if key < self.root.key:
```

```
    new_node.left = self.root.left
```

```
    new_node.right = self.root
```

```
    self.root.left = None
```

```
else:
```

```
    new_node.right = self.root.right
```

```
    new_node.left = self.root
```

```
    self.root.right = None
```

```
self.root = new_node
```

```
# Access file
```

```
def access(self, key):
```

```
    if self.root is None:
```

```
        return False
```

```
    self.root = self.splay(self.root, key)
```

```
return self.root.key == key
```

```
# Display inorder
```

```
def inorder(self, root):
```

```
    if root:
```

```
        self.inorder(root.left)
```

```
        print(root.key, end=" ")
```

```
        self.inorder(root.right)
```

```
# Display root
```

```
def show_root(self):
```

```
    if self.root:
```

```
        print("Current root:", self.root.key)
```

```
    else:
```

```
        print("Tree is empty")
```

```
tree = SplayTree()
```

```
n = int(input("Enter number of files:"))
```

```
print("Enter file names:")
```

```
for i in range(n):
```

```
    filename = input()
```

```
    tree.insert(filename)
```



```
print("\nFiles in cache:")
```

```
tree.inorder(tree.root)
```

```
print("\n")
```

```
# Access frequently used files
```

```
m = int(input("Enter number of file accesses: "))
```

```
for i in range(m):
```

```
    filename = input("Enter file to access: ")
```

```
    if tree.access(filename):
```

```
        print("File found and moved near root.")
```

```
    else:
```

```
        print("File not found.")
```

```
    tree.show_root()
```

Example

Suppose the cache contains:

main.c

data.c

functions.c

header.h

test.c

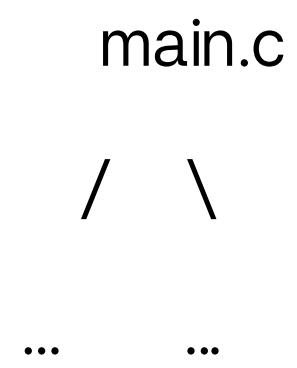
If the developer repeatedly accesses:

main.c

main.c

main.c

the Splay Tree moves main.c toward the root:



So subsequent accesses to main.c become efficient.

Virus Scan Problem

During a virus scan:

data.c

functions.c

header.h

main.c

test.c

each file is accessed once and then splayed toward the root.

This causes:

Access file



Splay operation



Tree restructuring



Next file



Another splay operation



More restructuring

Therefore, the previously useful cache structure can be disrupted.

Possible Improvement

A practical solution is to **detect sequential scans**. If many accesses are made in alphabetical order, temporarily disable splaying or use a separate scan mode:

Normal Access



Splay Tree



Frequently used files near root

Sequential Virus Scan Detected



Temporarily reduce/disable splaying



Preserve frequently accessed files

This is a good example of why **adaptive caching policies** can be better than blindly applying Splay Tree restructuring to every access.